CS4423 - Networks

Prof. Götz Pfeiffer
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

5. Power Laws and Scale-Free Graphs

Lecture 16: Power Laws

We have seen that **random graphs** have degree distributions that have the form of a **binomial distribution**, or, as the number of nodes increases, the form of a **Poisson distribution**. This means that most nodes in the graph have a degree that is equal or close to the average node degree of the graph. The probability that a node has a degree that is much higher than the average degree is so small that it can be neglected.

In **real world networks**, such as the WWW, the neural network that is the brain of C. Elegans, citation networks, protein interaction networks and many more, it has been observed that there are nodes of **extremely high degree**, compared to the average degree. Their number is small, but not negligible. The degree distibution of such a network is better described by a **power law**.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx
```

Degree Distribution

Recall the **degree distribution** of a random graph:

The **degree distribution** of an undirected graph $G = (X, E)$ is the function $k \mapsto p_k := n_k/n$, where $n = |X|$ and n_k is the number of nodes of degree k (and thus p_k is the probability that a random node $x \in X$ has degree k).

In an ensemble of graphs of order n, one sets $p_k := \overline{n_k}/n$, where $\overline{n_k}$ is the expected value of the random variable n_k over the ensemble of graphs.

In this sense, the degree distribution in a random $G(n, p)$ graph is **binomial** :

$$p_k = \binom{n-1}{k} p^k (1-p)^{n-1-k},$$

or, in the limit $n \to \infty$ and $p \to 0$ with np constant, it is a **Poisson distribution**:

$$p_k = e^{-z} \frac{z^k}{k!},$$

where $z = np$.

A power law degree distribution is strikingly different:

> A **power law** degree distribution has the form
> $$p_k = ck^{-\gamma},$$
> for $k \geq 1$ and certain constants c and γ. (Typically $2 \leq \gamma \leq 3$.)

Of course, the constant c needs to be chosen so that $\sum_k p_k = 1$. For simplicity, let's assume that $p_0 = 0$, i.e., that every node is connected to at least one other node, no node forms a disconnected singleton component.

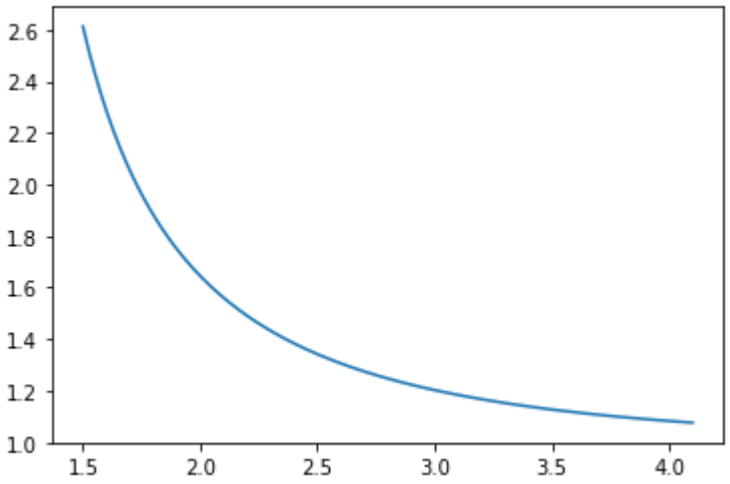
Then, since $\sum_{k=1}^{\infty} k^{-\gamma} = \zeta(\gamma)$, where ζ is the infamous Riemann zeta function (https://en.wikipedia.org/wiki/Riemann_zeta_function), we get $c = \zeta(\gamma)^{-1}$ and
$$p_k = \frac{k^{-\gamma}}{\zeta(\gamma)}.$$

The `scipy` package has a `zeta` function that can be used for plotting a graph over the domain of interest.

In [2]:
```
from scipy.special import zeta
```

In [3]:
```
x = np.linspace(1.5, 4.1, 200)
plt.plot(x, zeta(x))
```

Out[3]:
```
[<matplotlib.lines.Line2D at 0x7fe8b5b8d0d0>]
```



As a function of the real numbers, $\zeta(s)$ only converges for $s > 1$. Specific values include

- $\zeta(2) = \pi^2/6 \approx 1.6449$,
- $\zeta(3) \approx 1.2021$ aka Apéry's constant (https://en.wikipedia.org/wiki/Ap%C3%A9ry%27s_constant),
- $\zeta(4) = \pi^4/90 \approx 1.0823$.

In the limit, $\zeta(s) \to 1$ as $s \to \infty$.

Let's plot a binomial/Poisson distribution against some power law degree distribution.

In [4]:
```
def binomial(n, k):
    prd, top, bot = 1, n, 1
    for i in range(k):
        prd = (prd * top) // bot
        top, bot = top - 1, bot + 1
    return prd
```

In [5]:
```
def b_dist(n, p, k):
    return binomial(n, k) * p**k * (1-p)**(n-k)
```

In [6]:
```
from math import exp, factorial
def p_dist(l, k):
    return exp(-l) * l**k / factorial(k)
```

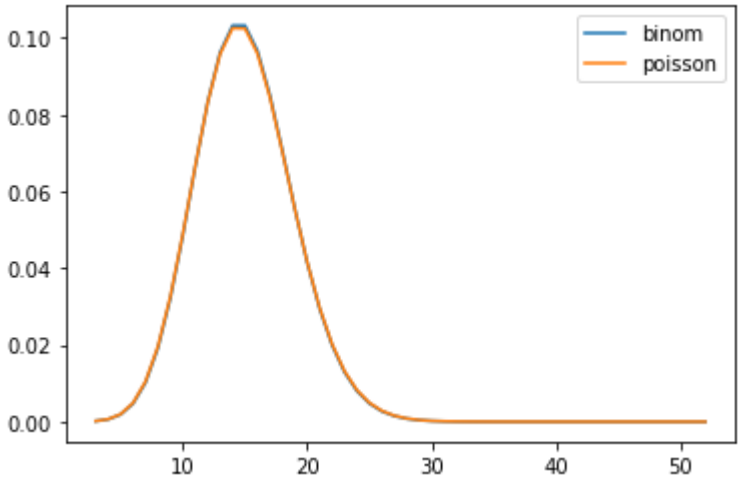
The specific values of $n = 1000$ and $p = 0.015$ correspond to a random graph with $n = 1000$ nodes and $m = 7493$ edges. Here, the **average degree** is $2m/n \approx 15$.

In [7]:
```
n, p = 1000, 0.015
domain = range(3, 53)
l = p * (n-1)
binom = [b_dist(n-1, p, k) for k in domain]
poisson = [p_dist(l, k) for k in domain]
```

We already know that the binomial distribution and the Poisson distribution with these parameters are almost identical:

In [8]:
```
df = pd.DataFrame({'binom': binom, 'poisson': poisson}, index=domain)
df.plot()
```

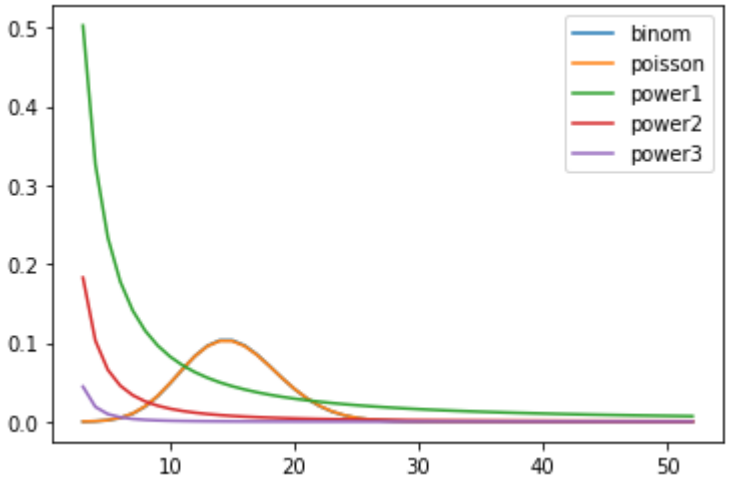
Out[8]: <AxesSubplot:>



In [9]:
```
def power_dist(c, gamma, k):
    return c * k**(-gamma)
```

In [10]:
```
df['power1'] = [power_dist(zeta(1.5), 1.5, k) for k in domain]
df['power2'] = [power_dist(zeta(2), 2, k) for k in domain]
df['power3'] = [power_dist(zeta(3), 3, k) for k in domain]
df.plot()
```

Out[10]: <AxesSubplot:>

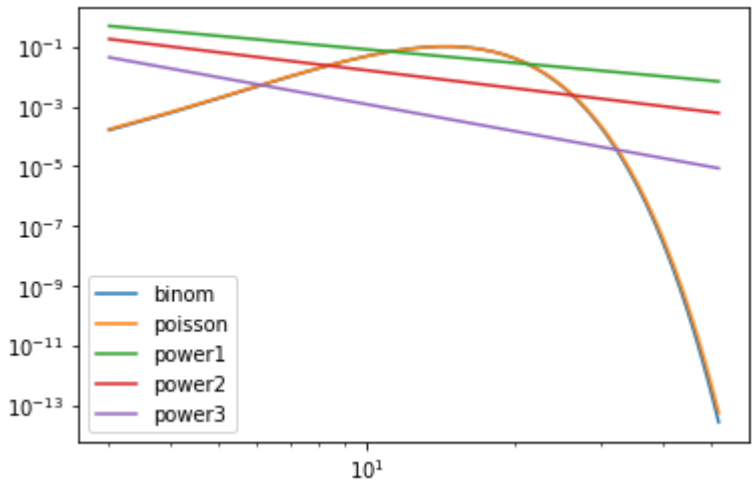


Splitting the plot at $k = 10$, say, on can distinguish the **long tail** of a power law distribution ($k > 10$) from the **dominating few** ($k < 10$), a phenomenon that is also known as the 80-20 rule (https://en.wikipedia.org/wiki/Pareto_principle).

The difference between the two types of distributions become even more apparent in a log-log plot (https://en.wikipedia.org/wiki/Log%E2%80%93log_plot).

In [11]:
```
df.plot(loglog=True)
```

Out[11]: <AxesSubplot:>



Note how the power laws appear as **straight lines** in this plot. Why?

Because $y = cx^{-\gamma}$ implies $\ln y = \ln c - \gamma \ln x$.

The Brain of Worm

Let `G` be the neural network that is the brain of C.Elegans.
Its network structure can be loaded from a file as follows.

In [12]:
```
G = nx.read_pajek("data/c_elegans_undir.net")
G = nx.Graph(G)
```

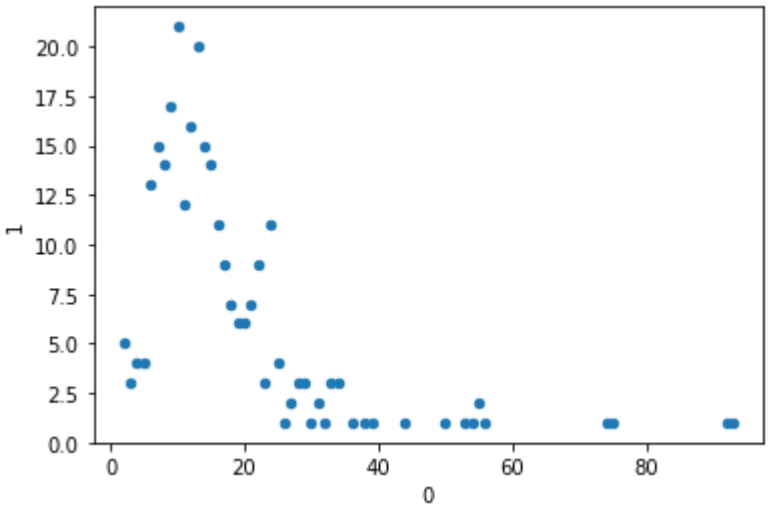
In [13]:
```
n, m = G.number_of_nodes(), G.number_of_edges()
n, m
```

Out[13]: (279, 2287)

Does `G` have a binomial degree distibution?

In [14]:
```
hist = nx.degree_histogram(G)
hist = [(k, p) for (k, p) in enumerate(hist) if p > 0]
df = pd.DataFrame(hist)
df.plot.scatter(x = 0, y = 1)
```

Out[14]: <AxesSubplot:xlabel='0', ylabel='1'>

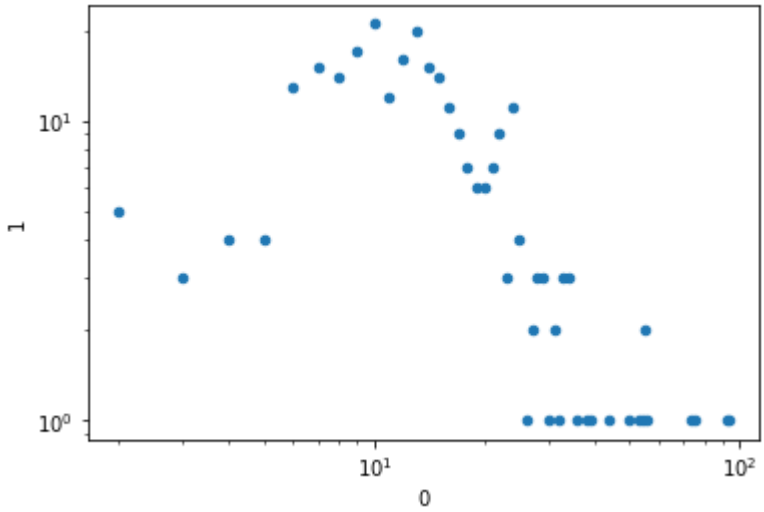


There definitely are some nodes of extremely high degree ...

Does the degree histogram of the worm brain network follow a power law degree distribution?
Here is a loglog plot of it ...

In [15]:
```
df.plot.scatter(x = 0, y = 1, loglog=True)
```

Out[15]: <AxesSubplot:xlabel='0', ylabel='1'>



Identifying a power-law behaviour, in particular extracting the exponent γ can be very tricky. It somehow requires fitting a straight line into the loglog plot of the degree distribution, paying special attention to the large values of k ...

Generating a Discrete Power Law Degree Distribution

Which **natural process** leads to a power law distribution? When the rich get richer ...

A growth process by **Random Copying** that results in a sequence of positive integers $(x_0, x_1, \ldots, x_l)$ adding up to m can be described as follows:

1. Start with $x_0 = 1$ and $l = 0$.
2. $m - 1$ times flip a fair coin:
   if it's heads, set $l \leftarrow l + 1$ and $x_l \leftarrow 1$
   if it's tails, pick k with probability $x_k / \sum x_i$ and set $x_k \leftarrow x_k + 1$.

In [16]:
```
import random
```

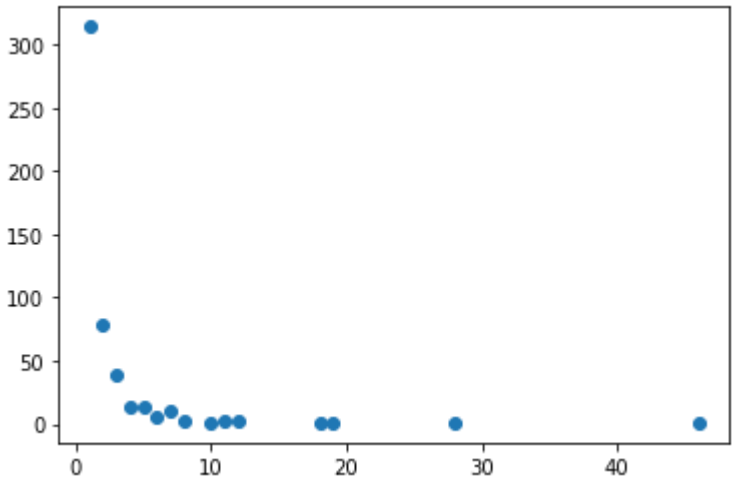
In [17]:
```
def powerlaw(m):

    # distribute m according to a power law
    x = [1]
    for i in range(m-1):
        p = random.random()
        if p < 0.5:
            x.append(1)
        else:
            k = random.choices(range(len(x)), x)[0]
            x[k] += 1

    # determine and return the distribution
    d = {}
    for a in x:
        d[a] = d.get(a, 0) + 1
    return d
```

Let's compute a power law and plot it.

In [18]:
```
p = powerlaw(1000)
```

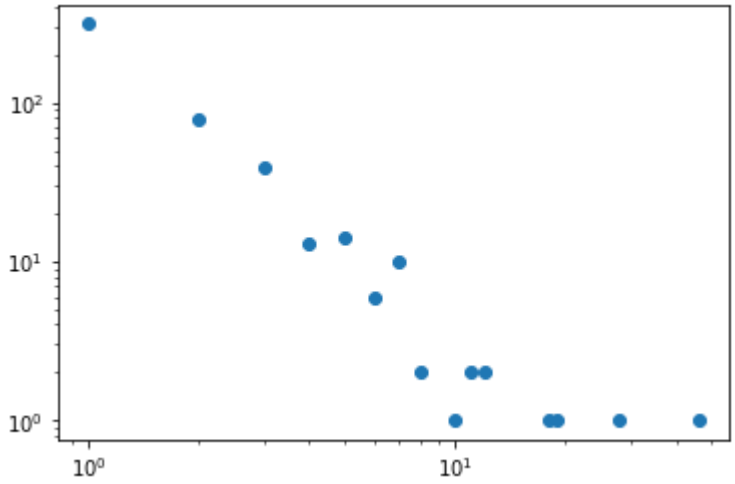
In [19]:
```
plt.scatter(p.keys(), p.values())
```

Out[19]:
```
<matplotlib.collections.PathCollection at 0x7fe8b135a750>
```



In [20]:
```
plt.loglog(p.keys(), p.values(), 'o')
```

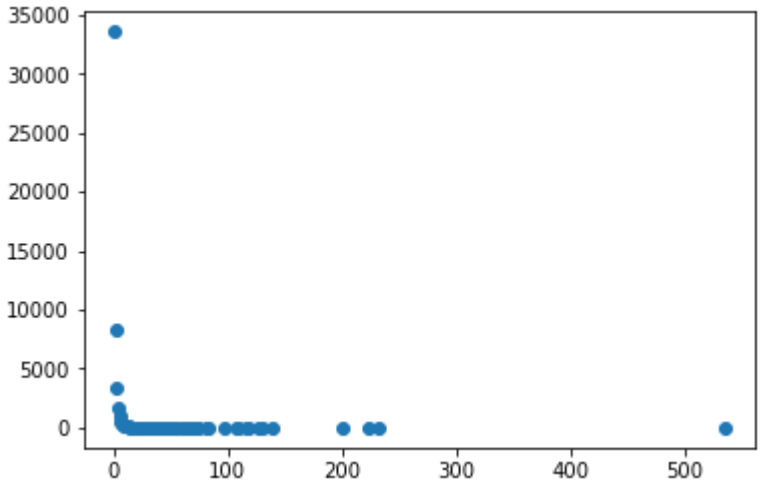
Out[20]:
```
[<matplotlib.lines.Line2D at 0x7fe8b12a2c10>]
```



In [21]:
```
p = powerlaw(100000)
```

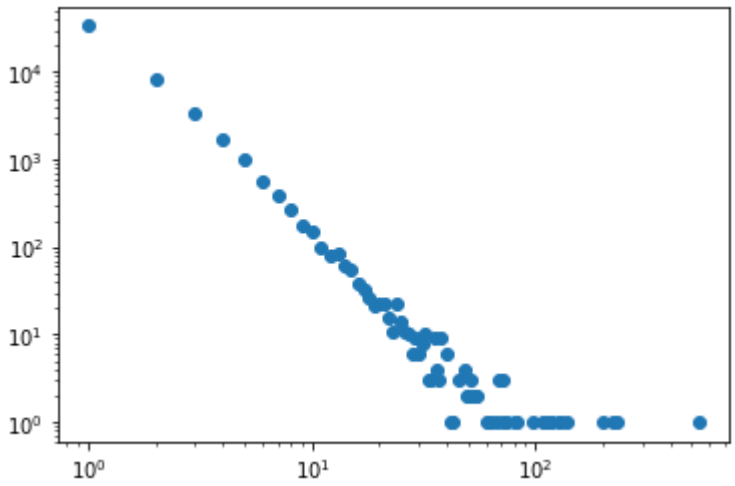
In [22]:
```
plt.scatter(p.keys(), p.values())
```

Out[22]:
```
<matplotlib.collections.PathCollection at 0x7fe8b11889d0>
```



In [23]:
```
plt.loglog(p.keys(), p.values(), 'o')
```

Out[23]:
```
[<matplotlib.lines.Line2D at 0x7fe8b10ba090>]
```

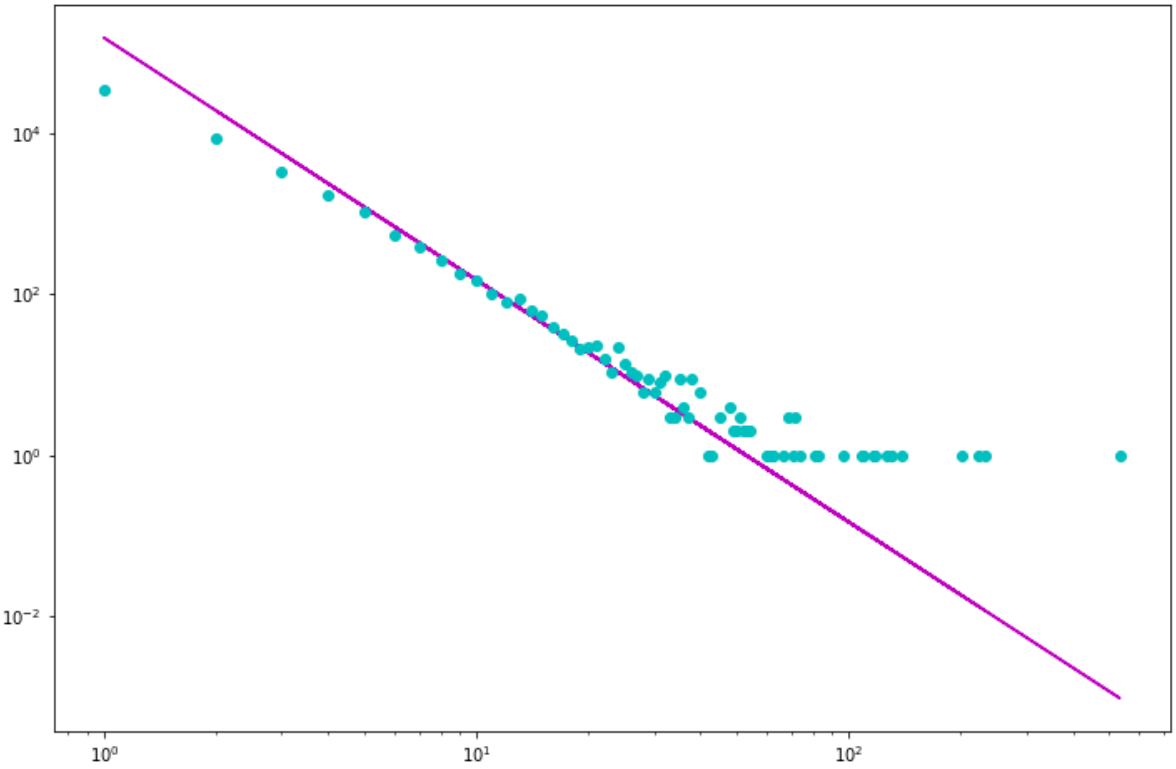


It can be shown that this process produces a power law distribution with $\gamma = 3$.

In [24]:
```
x = np.array(list(p.keys()))
y = np.array(list(p.values()))
```

In [25]:
```
plt.figure(figsize=(12,8))
plt.loglog(x, 15*10**4/x**3, '-m', x, y, 'oc')
```

Out[25]:
```
[<matplotlib.lines.Line2D at 0x7fe8b0f63710>,
 <matplotlib.lines.Line2D at 0x7fe8b0f63750>]
```



In fact, why use a fair coin? One can choose between increasing l or increasing one the existing x_k with any given probability p.

It can be shown that such a process produces a power law degree distribution with $\gamma = \frac{2-p}{1-p}$ ...

In [26]:
```
def powerlaw(m, p):

    # distribute m according to a power law
    x = [1]
    for i in range(m-1):
        if random.random() < p:
            x.append(1)
        else:
            k = random.choices(range(len(x)), x)[0]
            x[k] += 1

    # determine and return the distribution
    d = {}
    for a in x:
        d[a] = d.get(a, 0) + 1
    return d
```

In [ ]:
```
```

Code Corner

`random`

- `random` : [doc] (https://docs.python.org/3/library/random.html#random.random)
- `choices` : [doc] (https://docs.python.org/3/library/random.html#random.choices)

`matplotlib`

- `pyplot.plot` : [tutorial] (https://matplotlib.org/tutorials/introductory/pyplot.html)

`scipy`

- `special.zeta` : [doc] (https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.zeta.html)

Exercises

1. Compare the degree distribution of the worm brain network G to the degree distribution of a random graph R of the same order and size.
2. Compare the degree distribution of the worm brain network G to the degree distribution of a Watts-Strogatz W of the same order and size.
3. Try and plot a straight line into the loglog plot of the degree distribution of the worm brain network, in such a way that it illustrates the underlying power law.
4. More about Power Law degree distributions can be found in Chapter 5 of the book. Read through that chapter.

In [ ]: